tsis1Linux: Project Introduction and Machine Provisioning

Linux for DevOps and Cloud coursework, individual project

Ulanuly Alibi· 20 September 2026

1. Project Description

1.1 Legend

tsis1Linux is a small task-management service. Users register an account, log in, and keep a private list of todo tasks that they can create, read, update and delete through a REST API. The application itself is deliberately simple. It is the vehicle for the real subject of the coursework, which is running a service on Linux machines: provisioning servers, isolating the database from the internet, laying out the filesystem, deploying releases repeatably and keeping backups.

The scenario is the classic three-tier deployment. A reverse proxy receives public traffic and forwards it to the API, and the API stores everything in a PostgreSQL database that only the API machine can reach.

1.2 Main target

Deploy the service on two Linux machines so that it is secure by default, reproducible from scripts and updatable without manual file copying, and be able to demonstrate each of those properties.

1.3 Tasks

Task	Status
Choose and describe the project	Done (Section 1)
Design the architecture: machines, software units, automation plan	Done (Section 2)
Provision the machines; justify the platform and distribution	Done (Section 3)
Define storage places and script the filesystem preparation	Done (Section 4)
Deploy the first release and verify the whole chain	Done, manually (Section 5)
Automate rollout and updates with GitHub Actions	Planned (Section 2.4)
Encrypt public traffic with TLS	Planned (Section 6)
Copy backups off the database machine	Planned (Section 6)

1.4 The application

The API is written in Go 1.26 and uses only the standard library's `net/http` router. Data lives in PostgreSQL, accessed through `pgx`. Schema migrations from `golang-migrate` are embedded in the binary and applied on startup. Access tokens are short-lived JWTs (15 minutes by default), refresh tokens are rotated on every use, and passwords are hashed with bcrypt. Built with CGO disabled, the binary is a single self-contained file.

Configuration comes only from environment variables (`PORT`, `DATABASE_URL`, `JWT_SECRET`, `ACCESS_TOKEN_TTL`, `REFRESH_TOKEN_TTL`). The application does not read `.env` files, which makes a systemd `EnvironmentFile` a natural fit.

Method	Path	Auth	Description
POST	`/auth/register`	none	Create an account
POST	`/auth/login`	none	Get an access and refresh token pair
POST	`/auth/refresh`	none	Rotate a refresh token for a new pair
POST	`/auth/logout`	none	Revoke one refresh token
POST	`/auth/logout-all`	Bearer	Revoke every refresh token of the user
POST	`/tasks`	Bearer	Create a task
GET	`/tasks`	Bearer	List the user's tasks
GET	`/tasks/{id}`	Bearer	Get one task
PUT	`/tasks/{id}`	Bearer	Update a task
DELETE	`/tasks/{id}`	Bearer	Delete a task
GET	`/healthz`	none	Liveness check

1.5 Team

This is an individual project. The application, the infrastructure and the documentation are the work of one person, Alibi Ulanuly.

2. System Architecture

2.1 Overview

Three logical tiers run on two machines. `mainapp` is the only machine reachable from the internet and hosts nginx and the Go API. `main-db` hosts PostgreSQL and has no public address. The tiers are separated by process, user account and firewall rule rather than by extra virtual machines. A reverse proxy in front of a single API instance gains nothing from a machine of its own, whereas the database is the component whose exposure matters most, so it is isolated.

Public requests enter through the internet gateway on ports 80 and 443, pass the application's network security group and the host firewall, and reach nginx. nginx proxies them to the API on `localhost:8080`. The API opens a TLS connection to PostgreSQL on the private address of `main-db`, which accepts it only from the application machine. Administrators reach `mainapp` over SSH and `main-db` through it with ProxyJump, so no private key is ever stored on a server.

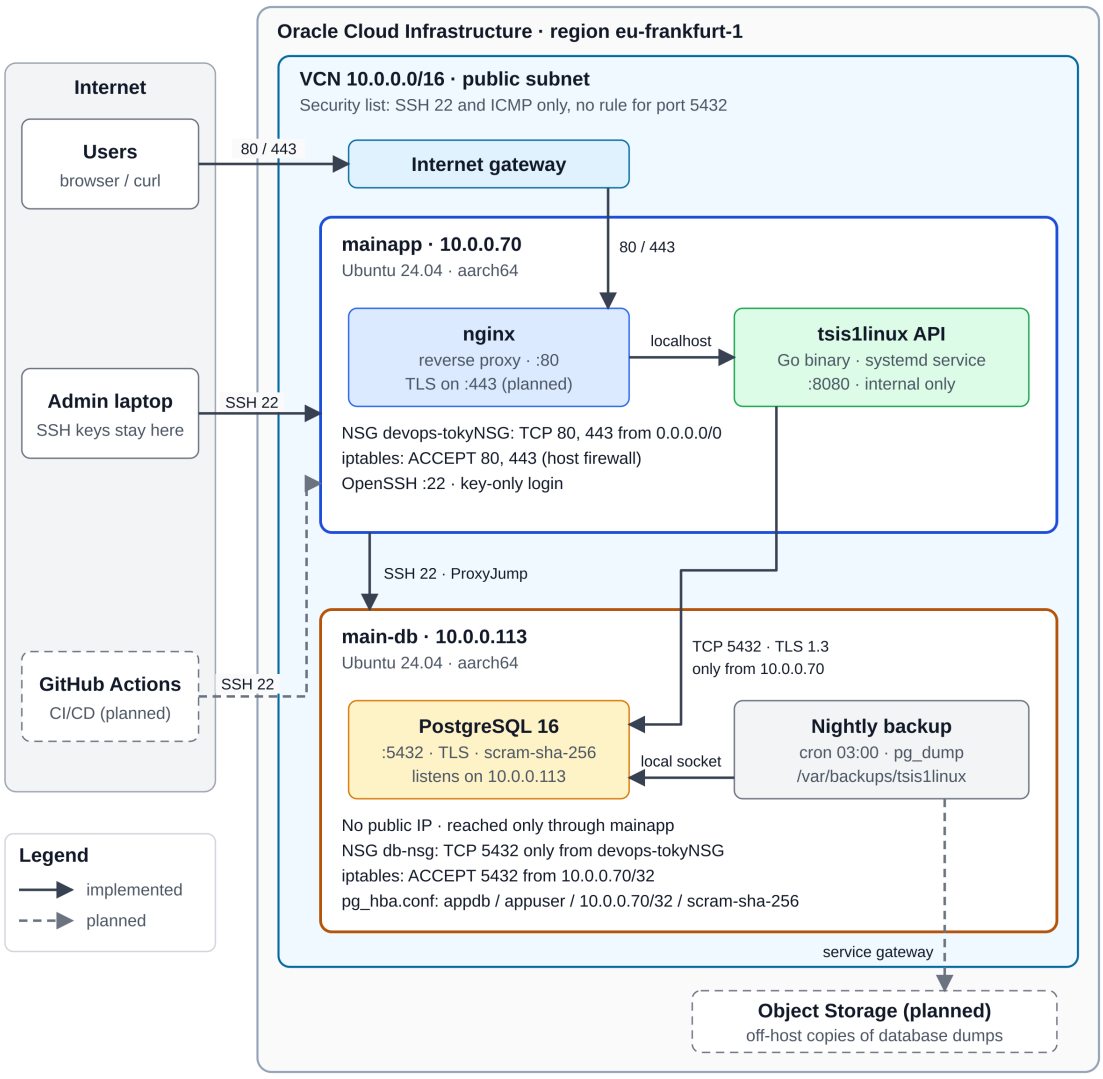


Figure 1. System architecture. Solid lines are implemented; dashed lines and boxes are planned.

2.2 Machines and software units

Machine	Role	Key software	Ports
`mainapp`	Application server	nginx (reverse proxy); tsis1linux API (Go binary run by systemd as an unprivileged user); OpenSSH; iptables with netfilter-persistent	22 and 80 public; 443 after TLS; 8080 local only
`main-db`	Database server	PostgreSQL 16; backup script and cron; OpenSSH; iptables with netfilter-persistent	22 (through `mainapp`); 5432 from `mainapp` only
GitHub (external)	Build and deployment (planned)	GitHub Actions workflow: build, test, deploy	Outbound SSH to `mainapp`

Neither server has a compiler or build tools. The binary is built elsewhere and only copied to `mainapp`.

2.3 Network and access control

A packet must be permitted at every layer to arrive. OCI evaluates security-list rules and network-security-group (NSG) rules additively, and separately from both the Ubuntu images ship with restrictive iptables rules. Each rule below therefore exists in its own place.

Layer	Applies to	Rule
OCI security list	The subnet (default list)	Ingress: TCP 22 from 0.0.0.0/0; ICMP types 3 and 4 from 0.0.0.0/0; ICMP type 3 from 10.0.0.0/16. No rule covers port 5432.
OCI NSG `devops-tokyNSG`	`mainapp`	Ingress: TCP 80 and 443 from 0.0.0.0/0.
OCI NSG `db-nsg`	`main-db`	Ingress: TCP 5432, stateful, with the source set to the NSG `devops-tokyNSG` instead of an address range.
Host firewall (iptables)	`mainapp`	ACCEPT for TCP 80 and 443, inserted above the default REJECT rule and saved with netfilter-persistent.
Host firewall (iptables)	`main-db`	ACCEPT for TCP 5432 from 10.0.0.70 only, above the default REJECT rule.
PostgreSQL	`main-db`	`listen_addresses = 'localhost, 10.0.0.113'`; `pg_hba.conf` lets `appuser` reach `appdb` from 10.0.0.70/32 with scram-sha-256. The API connects with `sslmode=require`; TLS 1.3 was observed.
SSH	Both machines	Key-only login with a separate key pair per machine. `main-db` is reached through `mainapp` with ProxyJump.

The database rule names the application's NSG as its source. Only instances that are members of that group can reach PostgreSQL, and a subnet-wide rule was deliberately avoided so that a future instance in the same subnet gets no database access by default.

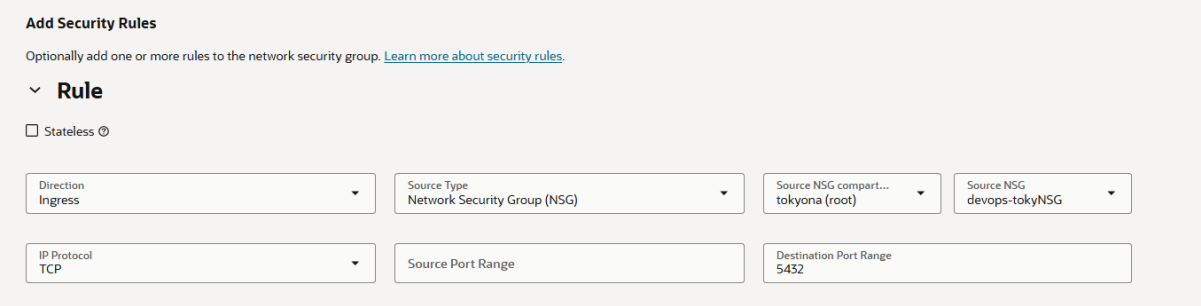


Figure 2. The single ingress rule of the database NSG.

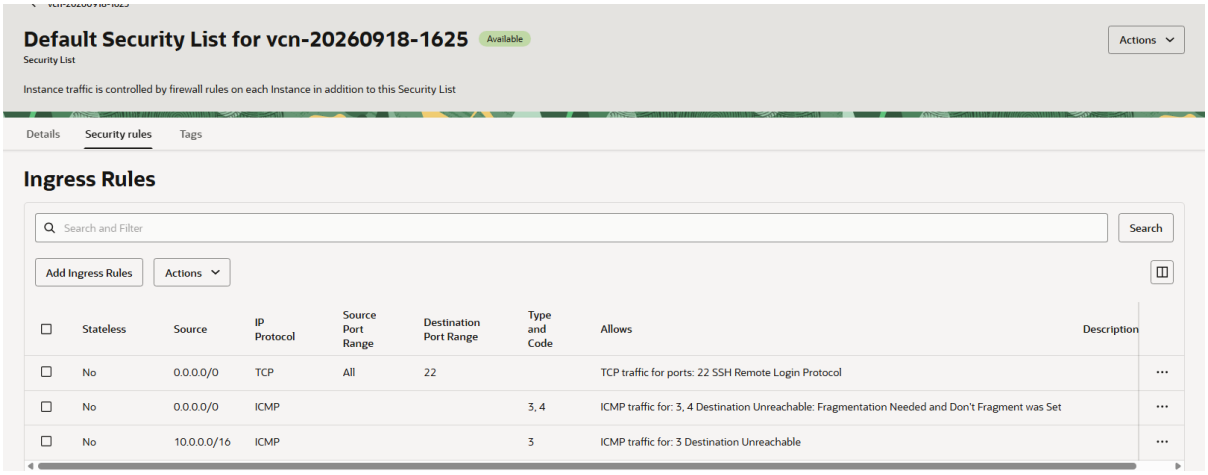


Figure 3. Ingress rules of the subnet's security list: no rule covers port 5432.

2.4 Automation plan: rollout and updates

Initial rollout. The first deployment was done by hand, once, to learn the failure modes before automating them.

1. Create the VCN networking, the two instances and the NSGs in the OCI console.
2. On `main-db`: install PostgreSQL 16, create the role `appuser` and database `appdb`, set `listen_addresses`, add the `pg_hba.conf` rule, restart, add the iptables rule and attach `db-nsg`.
3. On `mainapp`: install nginx.
4. On both machines: run `setup-fs.sh` with the role `app` or `db` (Section 4).
5. Fill in `/etc/tsis1linux/tsis1linux.env` on the server, so secrets never leave the machine.
6. Cross-compile the binary, upload it to `/opt/tsis1linux/releases/manual1/`, point the `current` symlink at it and enable the systemd service.
7. Enable the nginx site and verify the chain through the public address.

Updates with GitHub Actions (planned). The workflow already runs build, vet, gofmt, golangci-lint and tests on every push and pull request to `master`. A deploy job will follow it, run only on `master` and only when those checks pass:

1. Cross-compile with `CGO_ENABLED=0 GOOS=linux GOARCH=arm64 go build -o tsis1linux ./cmd/app`.
2. Copy the binary over SSH to `/opt/tsis1linux/releases/<commit-sha>/`, using a dedicated deploy key stored as a GitHub secret.
3. Repoint the symlink with `ln -sfn` and restart the service through a sudo rule limited to that one unit.
4. Check `/healthz` on the server. On failure, repoint the symlink to the previous release, restart, and fail the job.
5. Delete all but the last five releases.

Migrations are embedded in the binary and run on startup, so a deploy needs no separate migration step. Rolling the binary back does not roll the schema back, so every migration must stay compatible with the previous release.

Maintenance. Operating-system updates on `mainapp` use `apt` directly. `main-db` has no outbound route by design, so it needs a temporary public IP for updates until a NAT gateway is added (Section 6). Backups run nightly at 03:00 as `pg_dump` files in `/var/backups/tsis1linux`, keeping seven days. A dump was restored into a scratch database to prove it works (Section 5). Copying dumps to OCI Object Storage through a service gateway is planned.

3. Machine Provisioning

3.1 Platform: Oracle Cloud Infrastructure

The machines run on Oracle Cloud Infrastructure in the Frankfurt region (`eu-frankfurt-1`). There were three reasons. First, OCI exposes real network primitives (a VCN, security lists, network security groups, gateways), so isolating the database can be implemented and demonstrated rather than only described. Second, it offers ARM (Ampere) compute with an Always Free allowance, which is ample for a two-machine project. Third, the machines are real internet-facing servers, so public access, firewalls and deployment from an external CI service can be exercised realistically.

Local virtual machines (VirtualBox, Multipass) are free and simple but cannot demonstrate public exposure or cloud-level firewalls. A plain VPS is easier to set up but usually offers only a host firewall.

3.2 Distribution: Ubuntu Server 24.04 LTS

Both machines run Ubuntu Server 24.04 LTS. It receives standard security maintenance until 2029, so no operating-system upgrade is needed during the life of the project. PostgreSQL 16 and nginx are in the default repositories, so the database server is installed with one `apt` command and patched through the same channel. Ubuntu publishes maintained OCI images for both x86-64 and ARM64, and it is among the best-documented server distributions, which matters when troubleshooting.

The Go binary is self-contained, so the distribution affects only package management and the operating environment. Oracle Linux (RHEL-compatible, with firewalld and SELinux) was the main alternative and would also have worked. Ubuntu was preferred for its documentation and package availability.

3.3 Machines created

Property	mainapp	main-db
Role	Front-end and application server	Database server
Platform and region	OCI, `eu-frankfurt-1`	OCI, `eu-frankfurt-1`
Operating system	Ubuntu Server 24.04 LTS	Ubuntu Server 24.04 LTS
Architecture	aarch64 (ARM64)	aarch64 (ARM64)
Private IP	10.0.0.70	10.0.0.113

Property	mainapp	main-db
Public IP	Yes	None (attached temporarily for package installation)
NSG	`devops-tokyNSG`	`db-nsg`
Login	`ubuntu`, key only, own key pair	`ubuntu`, key only, own key pair

Two platform constraints shaped the setup. The instances are ARM64 (`uname -m` reports `aarch64`), so the API must be cross-compiled explicitly with `GOARCH=arm64`. And an instance without a public IP has no route out through the internet gateway, so `main-db` could not download packages until a public IP was attached for the installation.

3.4 Access configuration

SSH is configured on the administrator's laptop only. The `ProxyJump` line makes the connection to `main-db` pass through `mainapp`, which relays the encrypted stream and never sees a private key.

```
Host app
    HostName <app-public-ip>
    User ubuntu
    IdentityFile ~/.ssh/<app-key>
    IdentitiesOnly yes

Host db
    HostName 10.0.0.113
    User ubuntu
    IdentityFile ~/.ssh/<db-key>
    IdentitiesOnly yes
    ProxyJump app
```

3.5 Database server setup

```
sudo apt install postgresql
sudo -u postgres psql
  CREATE ROLE appuser WITH LOGIN;
  \password appuser
  CREATE DATABASE appdb OWNER appuser;

# /etc/postgresql/16/main/postgresql.conf
listen_addresses = 'localhost, 10.0.0.113'

# /etc/postgresql/16/main/pg_hba.conf
host    appdb    appuser    10.0.0.70/32    scram-sha-256

sudo systemctl restart postgresql
sudo iptables -I INPUT <n> -p tcp -s 10.0.0.70 --dport 5432 -j ACCEPT
sudo netfilter-persistent save
```

4. Filesystem Layout

The layout follows the Linux Filesystem Hierarchy Standard: software under `/opt`, configuration under `/etc`, variable data under `/var`. The table lists every place where the project stores anything. The API is stateless and writes no files, so there is no application data directory. Inventing one would only have filled a table cell.

Purpose	Path	Machine	Owner and mode	Notes
User data (accounts, tasks, tokens)	`/var/lib/postgresql/16/main`	main-db	`postgres:postgres` mode 700	Managed by the PostgreSQL package
Application data	none	mainapp	-	The API is stateless
Application binaries	`/opt/tsis1linux/releases/<version>/` `/opt/tsis1linux/current` (symlink)	mainapp	deploy user mode 755	One directory per release allows instant rollback
Application configuration and secrets	`/etc/tsis1linux/tsis1linux.env`	mainapp	`root:root` mode 600	Read by systemd as root; the directory is 750
Service definition	`/etc/systemd/system/tsis1linux.service`	mainapp	root mode 644	Runs as the unprivileged user `tsis1linux`
nginx site	`/etc/nginx/sites-available/tsis1linux`	mainapp	root mode 644	Enabled by a symlink in `sites-enabled`
PostgreSQL configuration	`/etc/postgresql/16/main/`	main-db	postgres and root (package defaults)	`postgresql.conf`, `pg_hba.conf`
Logs	journald (API) `/var/log/nginx/` (nginx)	mainapp	system defaults	No custom log directory needed
Backups	`/var/backups/tsis1linux/`	main-db	`postgres:postgres` mode 700	Nightly `pg_dump`, seven days kept
Backup job	`/usr/local/bin/tsis1linux-backup.sh` `/etc/cron.d/tsis1linux-backup`	main-db	root mode 755 and 644	Runs at 03:00 as `postgres`

`setup-fs.sh` creates this layout. It takes one argument, `app` or `db`, and must run as root. On `mainapp` it creates the unprivileged system user `tsis1linux` (no login shell, no home directory), the release and configuration directories, a sample environment file with placeholder secrets, the systemd unit and, if nginx is installed, a sample site configuration. On `main-db` it creates the backup directory, the backup script and the nightly cron entry.

The script is idempotent. Each file is created only if it does not already exist, so a second run never overwrites secrets that were edited by hand. Permissions are applied at creation time, so a secrets file is never briefly world-readable. The full listing is in Appendix A.

5. Verification

Check	Method	Result
The application machine reaches the database over TLS	`psql` from `mainapp` to 10.0.0.113 with `sslmode=require`	Connected; TLS 1.3, TLS_AES_256_GCM_SHA384
The API works end to end	`curl` to `/healthz`, `/auth/register` and `/auth/login` through the public address	All requests succeeded; the user was created and could log in
The database is not exposed	Reviewed the subnet security list; inspected the database VNIC	No rule covers port 5432; no public IP; only `db-nsg` attached

Check	Method	Result
Backups can be restored	Ran the backup script, restored the dump into a scratch database and listed its tables	Restored tables matched; scratch database dropped
The filesystem script is idempotent	Re-ran the script on `mainapp` after installing nginx	Existing files were left untouched

6. Known Limitations and Next Steps

Limitation	Impact	Planned mitigation
Public endpoint is plain HTTP	Passwords and tokens cross the network unencrypted	TLS in nginx with a Let's Encrypt certificate (needs a domain name)
Backups sit on the database machine's own disk	Protects against mistakes, not against loss of the disk or instance	Copy dumps to OCI Object Storage through a service gateway
`main-db` has no outbound internet	Security updates need a temporary public IP	Private subnet with a NAT gateway
The API's database role owns the schema	Migrations run at startup, so the running API holds DDL rights it does not need	Separate migration and runtime roles
SSH is open to the internet	Exposed to scanning; mitigated by key-only login	Restrict source ranges or use the OCI Bastion service, once deployment no longer needs hosted runners
Deployment is manual	Releases depend on the operator repeating the steps correctly	GitHub Actions deploy job (Section 2.4)

Appendix A. setup-fs.sh

Run as `sudo bash setup-fs.sh app` on `mainapp` and `sudo bash setup-fs.sh db` on `main-db`.

```
#!/usr/bin/env bash
# Creates the directory layout, service user and sample files for tsis1Linux.
# Safe to run repeatedly: existing files are never overwritten.
#
# Usage: sudo ./setup-fs.sh app    # on the app instance (nginx + API)
#        sudo ./setup-fs.sh db     # on the db instance (PostgreSQL)
#
# Optional: DEPLOY_USER=<user> sudo -E ./setup-fs.sh app
#           (the account CI/you use to upload releases; defaults to ubuntu)
set -euo pipefail

APP=tsis1linux
DEPLOY_USER="${DEPLOY_USER:-ubuntu}"
ROLE="${1:-}"

[[ $EUID -eq 0 ]] || { echo "run as root (use sudo)" >&2; exit 1; }
[[ $ROLE == app || $ROLE == db ]] || { echo "usage: $0 app|db" >&2; exit 1; }

# Create a file from stdin only if it does not exist yet.
# Permissions are applied at creation time, so secrets are never briefly world-readable.
write_if_missing() {
  local path=$1 owner=$2 group=$3 mode=$4
  if [[ -e $path ]]; then
    echo "skip     $path (already exists)"
    cat >/dev/null
  else
    install -m "$mode" -o "$owner" -g "$group" /dev/stdin "$path"
    echo "created  $path"
  fi
}

setup_app() {
  # Unprivileged account the API runs as: no login shell, no home directory.
  if ! id "$APP" &>/dev/null; then
    useradd --system --user-group --no-create-home --shell /usr/sbin/nologin "$APP"
    echo "created  system user $APP"
  fi

  # Binaries: one directory per release, "current" symlink points at the live one.
  install -d -m 755 -o "$DEPLOY_USER" -g "$DEPLOY_USER" "/opt/$APP" "/opt/$APP/releases"

  # Configuration: systemd reads the env file as root, so root-only is enough.
  install -d -m 750 -o root -g root "/etc/$APP"

  write_if_missing "/etc/$APP/$APP.env" root root 600 <<'EOF'
# Loaded by systemd via EnvironmentFile. The app does not read .env files itself.
PORT=8080
DATABASE_URL=postgres://appuser:CHANGE_ME@10.0.0.113:5432/appdb?sslmode=require
JWT_SECRET=CHANGE_ME
ACCESS_TOKEN_TTL=15m
REFRESH_TOKEN_TTL=168h
EOF

  write_if_missing "/etc/systemd/system/$APP.service" root root 644 <<'EOF'
[Unit]
Description=tsis1Linux todo API
After=network-online.target
Wants=network-online.target

[Service]
User=tsis1linux
Group=tsis1linux
EnvironmentFile=/etc/tsis1linux/tsis1linux.env
ExecStart=/opt/tsis1linux/current/tsis1linux
Restart=on-failure
RestartSec=2
```

```
NoNewPrivileges=yes
ProtectSystem=strict
ProtectHome=yes
PrivateTmp=yes

[Install]
WantedBy=multi-user.target
EOF
  systemctl daemon-reload

  if [[ -d /etc/nginx/sites-available ]]; then
    write_if_missing "/etc/nginx/sites-available/$APP" root root 644 <<'EOF'
server {
    listen 80;
    server_name _;   # replace with your domain when you set up TLS

    location / {
        proxy_pass http://127.0.0.1:8080;   # must match the port the API listens on
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
EOF
  else
    echo "note     nginx not installed; install it and re-run to get the sample site config"
  fi

  cat <<EOF

Next steps (app):
  1. Edit /etc/$APP/$APP.env and replace the CHANGE_ME values.
  2. Upload a build to /opt/$APP/releases/<version>/$APP and chmod 755 it.
  3. ln -sfn /opt/$APP/releases/<version> /opt/$APP/current
  4. systemctl enable --now $APP
EOF
}

setup_db() {
  id postgres &>/dev/null || { echo "postgres user not found; install PostgreSQL first" >&2;
exit 1; }

  # Backups: only the postgres user may read dumps.
  install -d -m 700 -o postgres -g postgres "/var/backups/$APP"

  write_if_missing "/usr/local/bin/$APP-backup.sh" root root 755 <<'EOF'
#!/usr/bin/env bash
set -euo pipefail
cd /
DIR=/var/backups/tsis1linux
pg_dump --format=custom --file="$DIR/appdb-$(date +%F).dump" appdb
find "$DIR" -name 'appdb-*.dump' -mtime +7 -delete
EOF

  write_if_missing "/etc/cron.d/$APP-backup" root root 644 <<EOF
# minute hour day month weekday user command
0 3 * * * postgres /usr/local/bin/$APP-backup.sh
EOF

  command -v cron &>/dev/null || echo "warning  cron is not installed; the nightly backup will
not run"

  echo
  echo "Data lives in /var/lib/postgresql/16/main and config in /etc/postgresql/16/main
(managed by the package)."
}

case $ROLE in
  app) setup_app ;;
  db)  setup_db ;;
esac
```